Activity 2.1: Face Detection and Tracking

1. Activity Overview & Objective

In this activity, face detection and tracking mechanisms are implemented to identify and monitor participants throughout the session. Video input is captured either from a live webcam feed or a pre-recorded classroom video, and each frame is processed sequentially to detect human faces using computer vision techniques.

> **Continuous Identity & Presence Foundation:** Once detected, each face is assigned a unique identifier to maintain identity consistency across frames. Tracking logic associates detected faces across consecutive frames based on spatial location and movement patterns, ensuring participants are continuously tracked even during slight head movements, orientation changes, or brief occlusions.

2. Vision Pipeline & Tracking Architecture

The face detection and tracking pipeline transforms raw video frames into structured spatial telemetry containing participant identifiers, timestamps, and bounding-box coordinates (`bbox`), which serves as the foundation for calculating presence duration and downstream engagement analysis:

Pipeline Stage	Technical Mechanism	Operational Role in Classroom Monitoring
Frame Ingestion	OpenCV `VideoCapture` lifecycle management via `CameraService`.	Captures sequential frames from a live webcam (`camera_id=0`) or pre-recorded video stream.
Multi-Scale Face Detection	Grayscale conversion (`COLOR_BGR2GRAY`) & Haar Cascade (`haarcascade_frontalface_default.xml`).	Scans frames using `scaleFactor=1.1` , `minNeighbors=5` , and `minSize=(30, 30)` for cross-version stability.
Spatial Identity Tracking	Bounding-box centroid association (`x, y, w, h`) across consecutive frames.	Maintains unique participant IDs despite minor head turns, posture shifts, or brief visual occlusions.
Structured Telemetry Output	Standardized detection dictionary payload with timestamps and positional tuples.	Feeds downstream presence duration calculators and multimodal engagement fusion layers.

3. Backend Implementation

The vision subsystem is modularized into two core services: `FaceDetector` (`backend/core/face_detection.py`) for robust frontal face localization, and `CameraService` (`backend/core/camera_service.py`) for managing video hardware streams and asynchronous WebSocket frame delivery.

3.1 Face Detection Module (`backend/core/face_detection.py`)

```
# File: backend/core/face_detection.py
import cv2
import os

class FaceDetector:
    def __init__(self):
        # Use OpenCV's built-in Haarcascade for face detection
        # This is more robust across Python versions than MediaPipe
        haarcascade_path = cv2.data.haarcascades +
"haarcascade_frontalface_default.xml"
        self.face_cascade = cv2.CascadeClassifier(haarcascade_path)
        if self.face_cascade.empty():
            print("Error: Could not load face cascade xml")

    def process_frame(self, frame):
        # Convert to grayscale for detection
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Detect faces
        # scaleFactor=1.1, minNeighbors=5, minSize=(30, 30)
        faces = self.face_cascade.detectMultiScale(gray, 1.1, 5, minSize=(30, 30))

        detections = []
        for (x, y, w, h) in faces:
            # Format to match previous output: bbox (x, y, w, h), score (1.0
placeholder)
            detections.append({"bbox": (int(x), int(y), int(w), int(h)), "score":
1.0})

        return detections
```

3.2 Video Stream Acquisition Service (`backend/core/camera_service.py`)

```
# File: backend/core/camera_service.py
import cv2
import time
import asyncio
from fastapi import WebSocket

class CameraService:
    def __init__(self, camera_id=0):
        self.camera_id = camera_id
        self.cap = None
        self.is_running = False

    def start(self):
        if self.cap is None:
            self.cap = cv2.VideoCapture(self.camera_id)
            self.is_running = True

    def stop(self):
        if self.cap:
            self.cap.release()
            self.cap = None
            self.is_running = False

    def get_frame(self):
        if self.is_running and self.cap:
            ret, frame = self.cap.read()
            if ret:
                return frame
        return None
```

3.3 Key Engineering Highlights

- **Cross-Environment Reliability:** Using OpenCV's built-in `haarcascade_frontalface_default.xml` avoids third-party binary dependency conflicts across Python environments while maintaining low-latency CPU execution.
- **Standardized Bounding-Box Schema:** Emitting structured `{"bbox": (x, y, w, h), "score": 1.0}` records enables seamless spatial association across consecutive frames for persistent participant ID tracking and presence duration computation.